BEACON

An AI engine that matches patients to clinical trials, and shows proof for every claim

Complete project guide: the idea, requirements, architecture, evaluation, build plan, learning roadmap and interview prep

Prepared for	Hasan · graduate student, aiming for AI engineer roles
Date	November 7, 2026 (version 1.0)
Core skills shown	Production RAG · AI agents · LLM evaluation · event-driven backend · observability · Docker + CI/CD
Benchmark	TREC Clinical Trials 2021 and 2022 (expert-labeled), compared with NIH's TrialGPT
Estimated effort	110–150 hours including learning · API budget about $30–80

Beacon is decision support for doctors and patients to discuss together. It is not medical advice, and it is built and evaluated only on synthetic patient cases.

Contents

1. The honest verdict: will this impress Big Tech? 3

2. The project in plain words 3

3. What already exists, and how Beacon is different 4

4. Requirements 5

5. System architecture 7

6. Tech stack: what each tool is and why it's here 9

7. The data 10

8. The RAG pipeline in depth 11

9. The agent layer (LangGraph) 12

10. Evaluation plan (what makes this project stand out) 13

11. Production patterns (the backend depth) 15

12. Observability 15

13. Cost plan and controls 15

14. Repository layout 16

15. Build plan 16

16. Learning roadmap: everything you need to learn 17

17. Testing, CI/CD and containers 20

18. Resume, README and interview prep 20

19. Risks and how to handle them 22

20. Medical, safety and ethics notes 22

Appendix A. Glossary in plain words 22

Appendix B. Where to learn each thing 24

1. The honest verdict: will this impress Big Tech?

Short answer: yes, if you finish it and measure it. A finished Beacon, with real benchmark numbers, clean code, tests, CI and a short demo, is in the top few percent of student portfolio projects. Most student AI projects are PDF chatbots with no measurements. Beacon is evaluated on an official, expert-labeled benchmark and compared with a published NIH system, and very few student projects can claim that.

What it will do for you

- **Get you past resume screens.** It names the skills that automated filters and recruiters look for (RAG, LLM agents, vector databases, evaluation, Kafka, Redis, PostgreSQL, Docker, CI/CD), each with a number attached.
- **Give you a great interview story.** "Tell me about a project" is asked in almost every interview. You'll have a real problem, real tradeoffs, real numbers and real failures to discuss.
- **Make you an obvious fit for health-AI teams** (Tempus, Triomics, Google's health teams, Microsoft) while staying impressive for any AI engineer role.

What it will not do (read this twice)

- **It does not replace coding interviews.** Big Tech internship interviews are mostly data structures and algorithms (LeetCode-style). Keep about 1 hour a day of practice going while you build. The project gets you the interview; algorithms get you through it.
- **An unfinished project hurts.** A maze of half-working services with no numbers looks worse than a small finished project. The working core plus the evaluation numbers plus the README come before every extra feature.
- **You must be able to explain every important line.** Use AI help to learn faster, but type and understand the retrieval, evaluation and agent code yourself. Project deep-dive interviews quickly expose code you can't explain.
- **Other things still count:** your school, past experience and referrals. Apply by November 15 no matter what state the project is in. Listing it as "in progress" with a GitHub link is completely fine.

Realistic time

If most of these tools are new to you, plan on **110–150 hours including learning**. The 8-day sprint in Section 15 gets you a working core with real numbers by November 14. Polish and extensions take another 2–4 weeks at a normal pace.

2. The project in plain words

What is a clinical trial?

A clinical trial is a study that tests a new treatment, such as a new cancer drug, before it is approved. For a patient whose normal treatment stopped working, a trial can be the best option left. Every trial has strict **eligibility rules**: *inclusion criteria* (you must have these) and *exclusion criteria* (you must not have these). Break one rule and you cannot join.

The problem, as a story

Sarah is 58. She has lung cancer, and her standard chemotherapy stopped working. Somewhere there is a trial she could join, but ClinicalTrials.gov lists more than 500,000 studies, each with 20–50 dense rules, and her doctor has 15 minutes per appointment. So nobody checks. Sarah never hears about the trial, and the trial struggles to find patients like her. Slow recruitment is one of the biggest reasons trials run late or fail.

What Beacon does

1. **Reads the patient's record** (a doctor's note) and turns it into a structured profile: age, diagnosis, mutations, past treatments, lab results and location.
2. **Searches every trial** with hybrid retrieval (exact keywords plus meaning) and narrows 375,000 trials to about 20 strong candidates. This is the RAG core.
3. **Checks every rule of every candidate** against the record. Each answer (met / not met / not enough information) must cite the exact sentence of the patient note that proves it.
4. **Ranks the trials and explains them**: a detailed view for the doctor, a plain-language summary for the patient, and a list of the missing facts that would settle the open questions.

Example: one trial checked for Sarah (illustrative)

Trial rule	Sarah	Proof from her record
Age 18 or older	Met	Sentence 1: “58-year-old woman…”
Confirmed EGFR mutation	Met	Sentence 2: “…EGFR exon 19 deletion”
Chemotherapy finished at least 4 weeks ago	Met	Sentence 4: “finished chemotherapy in March”
No brain metastases	Not enough information	Not mentioned: ask for a brain MRI result
Adequate kidney function	Met	Sentence 6: “creatinine within normal limits”

> **What Beacon is (and is not)**
> Beacon never diagnoses and never decides. It does the hours of searching and reading so the doctor and the patient can decide together. Every claim points to evidence, which is how the system controls hallucination.

3. What already exists, and how Beacon is different

Clinical-trial matching is a real, funded field, which proves it matters:

- **Companies:** Triomics (Mount Sinai's cancer center rolled out its PRISM platform system-wide in January 2026), Dyania (Cleveland Clinic, 2026 Gartner innovation award), AIwithCare (a Mass General Brigham spin-out), Tempus (acquired Deep 6 AI), Antidote, myTomorrows and Massive Bio.
- **Research:** NIH's TrialGPT (Nature Communications, 2024) reported 87.3% criterion-level accuracy and 40% less screening time for clinicians, but it is still a research prototype. Also LLM-Match and TrialMatchAI (2025).

What there is **not** is a pile of student clones. Recruiters see hundreds of PDF chatbots and almost no trial-matching systems. Beacon's angles:

Beacon's angle	Why it matters
Open source, publicly benchmarked	Company tools are closed; nobody outside can check their accuracy. Yours is reproducible on TREC.
Evidence for every rule	Every judgment cites patient-note sentences, and a verifier measures how often the evidence really supports it.
Production-grade system	Research code is scripts. Beacon is a durable, observable, event-driven service with tests and CI.
Built for patients too	Plain-language summaries (and later, other languages) instead of hospital-only tools.
Cost-aware	You report cost per patient, which companies care about and student projects almost never measure.

4. Requirements

4.1 Functional requirements (what it must do)

ID	Requirement
FR-1	Accept a patient description (free-text note) through a REST API and the web console.
FR-2	Extract a structured patient profile (age, sex, conditions, biomarkers, treatments, labs) with validated JSON output.
FR-3	Retrieve candidate trials with hybrid search (BM25 + dense vectors), RRF fusion, age/sex filters and cross-encoder reranking.
FR-4	Split each trial's eligibility text into individual inclusion and exclusion rules and cache them.
FR-5	Judge every rule as met / not met / insufficient information / not applicable, citing patient-note sentence numbers.
FR-6	Verify evidence: flag any judgment whose cited sentences do not support it.
FR-7	Rank trials (likely eligible → uncertain → excluded) and list the missing facts per trial.
FR-8	Generate a plain-language patient summary and a detailed clinician view.
FR-9	Stream live progress to the console (server-sent events).
FR-10	Record every LLM call (model, tokens, cost, latency) and every judgment for audit and replay.
FR-11	Run the full TREC evaluation with one command and write a report.
FR-12	Accept reviewer feedback (“this judgment is wrong”) and store it as future evaluation data.

4.2 Non-functional requirements (targets you measure)

Area	Target (measure it; don't assume it)
Retrieval latency	p95 under 500 ms per query over 375K trials on your laptop
End-to-end match	p95 under 90 s for one patient across the top 20 trials (parallel LLM calls)
Cost	Under $0.05 per patient in development mode (Claude Haiku 5.5)
Grounding	Every judgment has a valid citation or is labeled “insufficient information”
Reliability	Zero lost requests when a worker is killed mid-job; duplicate messages never create duplicate results
Reproducibility	One command starts everything; the evaluation report regenerates from a clean clone

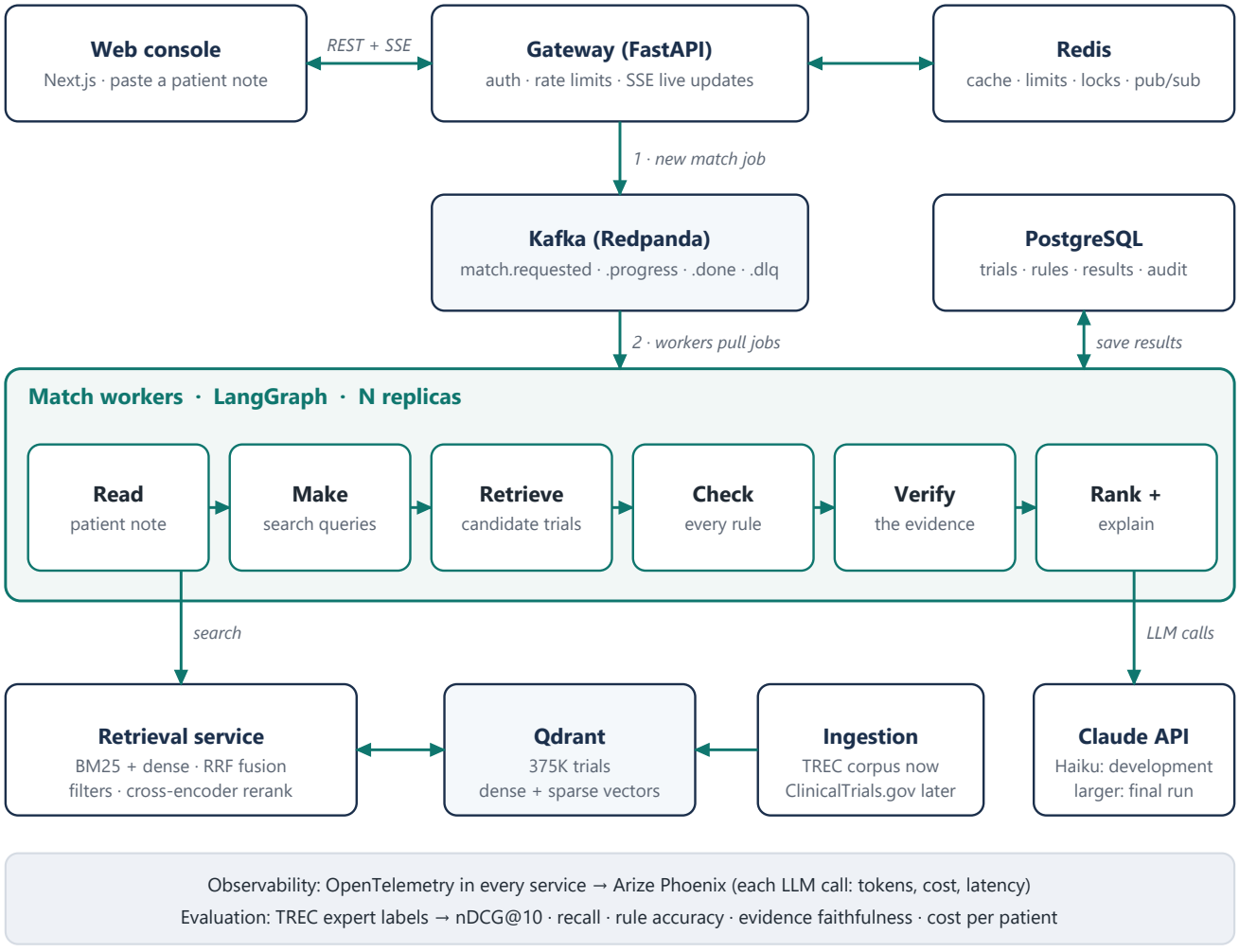
Area	Target (measure it; don't assume it)
Quality gates	CI runs lint, type checks, tests and a small evaluation on every pull request
Safety	Synthetic data only; clear “not medical advice” notices; no real patient data stored

4.3 What you need before starting

Item	Details
Computer	Windows 11 with WSL2 (Ubuntu). 16 GB RAM recommended (8 GB works if you run fewer services). About 25 GB free disk.
Software	Docker Desktop (WSL2 backend), Git, Python 3.12, uv, Node.js 20+, VS Code. Develop inside WSL2: Docker and Python tooling are much smoother there.
Accounts	GitHub (code, CI, image registry). Anthropic Console account with an API key. Set a monthly spend limit (for example $50) on day one.
Optional	Google Colab (free GPU) if you want to try the biomedical embedding model MedCPT later.
Budget	About $30–80 in API costs for the whole project (see Section 13).

5. System architecture

Beacon is a small set of services. Each one exists for a reason you can defend in an interview: different scaling needs, different failure modes, or a different speed requirement.



Observability: OpenTelemetry in every service → Arize Phoenix (each LLM call: tokens, cost, latency)

Evaluation: TREC expert labels → nDCG@10 · recall · rule accuracy · evidence faithfulness · cost per patient

5.1 Components and why each is separate

Component	Job	Why it's its own piece
Gateway (FastAPI)	Public API, auth, rate limits, starts match jobs, streams progress	Must stay fast and responsive; scales with the number of users
Kafka (Redpanda)	Durable job queue and event log between the gateway and the workers	Jobs survive crashes, can be replayed, and spread across many workers
Match workers (LangGraph)	Run the multi-step agent pipeline per patient	Long-running (seconds to minutes); scale with queue length
Retrieval service (FastAPI)	Hybrid search plus reranking	CPU-heavy reranker; used by workers and later by the MCP server
Qdrant	Stores dense and sparse vectors for 375K trials	Purpose-built vector search with filters

Component	Job	Why it's its own piece
PostgreSQL	Trials, parsed rules, patients, results, audit log, evaluation runs	The source of truth; transactions and SQL queries
Redis	Rate limits, exact-match LLM cache, locks, live-update pub/sub	Microsecond lookups for hot data
Ingestion job	Loads trials into Postgres and Qdrant	Batch work that runs rarely; must not slow user traffic
Phoenix + OpenTelemetry	Traces of every request and LLM call, with tokens and cost	See what the AI did, how long it took and what it cost
Web console (Next.js)	Demo UI	Kept deliberately thin

5.2 One request, step by step

1. The console sends the patient note to `POST /v1/matches`. The gateway checks the API key and rate limit (Redis), stores the request in Postgres and returns a `match_id` right away (HTTP 202).
2. The gateway publishes a `match.requested` event to Kafka. In the same database transaction it records the event in an *outbox* table, so the request and the event can never get out of sync.
3. A match worker picks up the job. LangGraph runs the pipeline: profile the patient, generate search queries, retrieve, check rules, verify evidence, rank and explain. It saves a checkpoint after each step.
4. Every step publishes a `match.progress` event. The gateway relays it to the browser through Redis pub/sub and server-sent events, so the user watches results appear live.
5. Results are written to Postgres and a `match.completed` event is published, which the evaluation tooling can consume.
6. If a worker crashes, Kafka redelivers the job and the next worker resumes from the last checkpoint. Writes are idempotent, so a repeat never duplicates results.

5.3 Data model (PostgreSQL)

Table	Holds
trials	NCT ID, title, conditions, interventions, phase, status, min/max age, sex, locations, raw eligibility text
trial_criteria	One row per rule: trial ID, inclusion/exclusion, position, text (parsed once, reused for every patient)
patients	Synthetic patient notes, split into numbered sentences, plus the extracted profile (JSON)
match_runs	One row per request: status, timings, total tokens, total cost, config version
trial_matches	Per patient-trial pair: retrieval rank, final score, eligibility category, summary
criterion_judgments	Per rule: label, cited sentence IDs, explanation, verifier verdict
llm_calls	Audit log: model, prompt hash, input/output/cached tokens, cost, latency, request ID
feedback	Reviewer corrections, which become future evaluation data
eval_runs	Benchmark results per configuration, so you can compare runs over time
outbox	Events waiting to be published to Kafka (transactional outbox pattern)

5.4 Kafka topics and Redis keys

Kafka topic	Purpose
match.requested	New match jobs; message key = match_id so all events of one job stay in order
match.progress	Step-by-step updates for the live console
match.completed	Finished results, for evaluation and analytics
match.retry / match.dlq	Retry with delay; after N failures the job goes to the dead-letter queue for inspection
trials.updated (phase 2)	Daily new or changed trials from ClinicalTrials.gov, re-indexed incrementally

Redis key pattern	Purpose
rl:{api_key}	Token-bucket rate limiter, implemented as an atomic Lua script
idem:{key}	Idempotency keys, so a retried POST doesn't start a second job
llm:{sha256}	Exact-match cache of LLM responses (same prompt means a free answer)
lock:ingest	Only one ingestion job runs at a time
chan:match:{id}	Pub/sub channel feeding server-sent events to whichever gateway replica holds the browser connection

6. Tech stack: what each tool is and why it's here

Tool	In plain words	Its job in Beacon	Depth
Python 3.12 + uv	The language; uv installs packages and manages the monorepo fast	Everything backend	Deep
FastAPI	Framework for building web APIs in Python	Gateway and retrieval service	Deep
Claude API	The LLM (Haiku 5.5 for development, a bigger model for final numbers)	Profiles, queries, rule checks, summaries, judges	Deep
LangGraph	Library for multi-step AI workflows with saved state	The match pipeline with checkpoints and retries	Deep
Qdrant + FastEmbed	Vector database, plus local, free embedding models	Hybrid search over 375K trials	Deep
PostgreSQL	The standard relational database	Source of truth, audit log, results	Medium
Redis	Very fast in-memory database	Rate limits, cache, locks, live updates	Medium
Kafka (Redpanda)	Durable message stream between services (Redpanda is a Kafka-compatible single container)	Job queue, progress events, retries, DLQ	Medium
OpenTelemetry + Phoenix	Standard tracing, plus a UI built for LLM apps	See every step, token and dollar	Medium
Docker Compose	Runs all services with one command	Local deployment	Medium
GitHub Actions + pytest	Automated tests on every push	CI with an evaluation gate	Medium
MCP server	Lets AI assistants like Claude use your tools	Exposes search_trials / check_eligibility (phase 2)	Light

Tool	In plain words	Its job in Beacon	Depth
Next.js	React web framework	Thin demo console	Light

Deliberately cut (and why)

Kubernetes and KEDA (Docker Compose is enough), Drain3, Toxiproxy, semantic caching, Prometheus and Grafana (Phoenix covers the AI-specific observability), LangChain chains and LlamaIndex (you write the retrieval yourself so you understand and can defend it), and Pinecone (it can't run locally and hides the internals). Any of these can be added later if a job description asks for it.

> **Key design choice**
> Use LangGraph for orchestration, but call the Anthropic SDK directly inside each step. You keep full control of prompts, caching, token accounting and structured outputs, and interviewers can't catch you on something a framework did for you.

7. The data

7.1 TREC Clinical Trials 2021 and 2022 (your benchmark)

TREC is a long-running research competition run by NIST. The Clinical Trials track gives you three things:

- **A corpus:** a snapshot of ClinicalTrials.gov with about 375,000 trials (the 2021 and 2022 tracks share it).
- **Topics:** synthetic patient descriptions written to read like real admission notes. There are 75 in 2021 and 50 in 2022, so there is no privacy issue.
- **Relevance labels ("qrels"):** for each patient, experts judged many trials as **eligible (2)**, **excluded (1)** or **not relevant (0)**. This is your answer key.

Load it with the Python library `ir_datasets`, which downloads the corpus, topics and labels and gives you iterators. Check ir-datasets.com for the exact dataset IDs; they live under the clinicaltrials collection.

> **Important: use 2021 for tuning, 2022 for testing**
> Tune everything (prompts, fusion weights, top-k, scoring) on TREC 2021, and report your headline numbers on TREC 2022. That's a proper train/test split: it shows you didn't overfit to the test set, and interviewers notice.

7.2 ClinicalTrials.gov API v2 (live mode, phase 2)

The public ClinicalTrials.gov API returns current trials as JSON, including recruiting status and site locations with coordinates. In phase 2, a daily sync job publishes new and changed trials to `trials.updated`, and ingestion re-indexes only what changed. That turns Beacon from a benchmark system into a live one (for example, "only trials recruiting within 100 km").

7.3 Laptop-friendly indexing plan

What	How	Size and time (estimates)
Sparse (BM25)	FastEmbed Qdrant/bm25 over title + conditions + interventions + summary + eligibility text	Fast; small on disk
Dense	One overview vector per trial (title, conditions, interventions, brief summary) with BAAI/bge-small-en-v1.5 (384 dimensions)	About 0.6 GB of vectors; about 30–90 min on CPU, so run it overnight the first time
Structured fields	Age range, sex, phase, status and locations go to Postgres (and the Qdrant payload for filters)	Seconds per thousand trials
Rules	Inclusion/exclusion split per trial, done lazily for retrieved trials and cached in Postgres	Grows as you use it

8. The RAG pipeline in depth

8.1 Why hybrid search matters here

Medical text has two kinds of matches. **Exact tokens** such as gene names (EGFR, BRCA1), lab names (HbA1c, eGFR) and drug names must match exactly, which BM25 (keyword search) does well and embeddings often blur. **Meaning**, such as "kidney function normal" matching "adequate renal function", is what dense embeddings are good at. Running both and fusing the results beats either alone, and your ablation table will prove it with numbers.

8.2 Query side, step by step

1. **Patient profiling.** The LLM turns the note into a validated JSON profile (structured outputs with a Pydantic schema).
2. **Query generation.** The LLM writes a ranked list of 5–10 search queries (main diagnosis, key mutations, prior treatments), inspired by TrialGPT's retrieval stage.
3. **Hybrid retrieval per query.** Qdrant runs dense and sparse searches; results are fused with Reciprocal Rank Fusion (RRF).
4. **Fuse across queries** with RRF again, weighting earlier (more important) queries higher.
5. **Hard filters:** age and sex from the structured fields. Measure this: wrong parsing could drop good trials.
6. **Rerank** the top 100 with a cross-encoder (for example BAAI/bge-reranker-base), then keep the top 20 for rule checking.

```
def rrf(rankings: list[list[str]], k: int = 60) -> list[str]:
    """Reciprocal Rank Fusion: score(doc) = sum over rankings of 1 / (k + rank)."""
    scores: dict[str, float] = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
```

8.3 Rule-level matching (the heart)

The patient note is split into numbered sentences. For each candidate trial, one LLM call receives the numbered note plus that trial's list of rules, and must return one judgment per rule:

```
class CriterionJudgment(BaseModel):
    criterion_id: str
    kind: Literal["inclusion", "exclusion"]
    label: Literal["met", "not_met", "insufficient_info", "not_applicable"]
    evidence_sentence_ids: list[int]   # must point at real sentences in the note
    explanation: str                   # one or two sentences, for the clinician view
```

- **Citations are checkable.** Code verifies that every cited sentence ID exists. A second, cheaper check (the verifier) asks whether the cited sentence actually supports the label.
- **"Insufficient information" is a feature, not a failure.** It is how Beacon avoids guessing, and it feeds the "missing facts" list.
- **Scoring:** a trial is *excluded* if any inclusion rule is not met or any exclusion rule is met; otherwise it is ranked by the share of rules confirmed, then by retrieval score. Tune the formula on 2021 only.

8.4 Ablations to run (your proof table)

Configuration	What it tests
BM25 only	Classic keyword baseline
Dense only (bge-small)	General-purpose embeddings baseline
Hybrid (RRF)	Does combining help?
Hybrid + LLM query generation	Does rewriting the patient note into queries help recall?
+ cross-encoder rerank	Does reranking improve the top 10?
+ age/sex filters	Do hard filters help, or do they wrongly drop trials?
+ LLM rule-level matching	The full system, scored on graded TREC labels
Phase 2: MedCPT embeddings	Does a biomedical model beat a general one?
Phase 2: rule-level chunks with contextual headers	Does indexing individual rules (prefixed with the trial title and condition) help?

9. The agent layer (LangGraph)

Node	Does	Model / tool
profile_patient	Note → numbered sentences → structured profile	Claude, structured output
generate_queries	Profile → ranked search queries	Claude, low effort
retrieve	Queries → top 20 trials (calls the retrieval service)	HTTP tool with timeout and retry
parse_rules	Trial eligibility text → list of rules (cached per trial)	Rule-based splitter; LLM fallback for messy text
check_rules (fan-out)	One parallel task per trial → rule judgments with citations	Claude, parallel via LangGraph Send
verify	Checks citations exist and support each label; re-asks once if not	Code plus a cheap LLM check

Node	Does	Model / tool
rank_and_explain	Score, rank, missing-facts list, patient and clinician summaries	Code plus Claude

Reliability inside the graph

- **State** is a typed object (Pydantic) holding the profile, candidates, judgments, token counts and cost so far.
- **Checkpoints** are stored in Postgres after every node, so a crashed worker's job resumes where it stopped.
- **Budgets:** each match has a token and dollar ceiling. If it is reached, Beacon returns partial results clearly marked, never a silent failure.
- **Errors:** every tool call has a timeout and retries with exponential backoff plus jitter. If one trial's check fails twice, that trial is marked "could not evaluate" and the others continue.
- **Human in the loop:** reviewers can mark any judgment wrong in the console, and those corrections become evaluation data.

Claude API practices that matter here

- **Structured outputs:** use `client.messages.parse()` with Pydantic models instead of parsing JSON by hand.
- **Prompt caching:** order each request as stable instructions → the numbered patient note (cache breakpoint) → this trial's rules. The same patient is checked against 20 trials, so the shared prefix is cached and much cheaper. Confirm it is working with `usage.cache_read_input_tokens`. Very short prefixes won't cache (the minimum depends on the model).
- **Batches API:** full benchmark runs go through Message Batches at 50% off. Results come back in any order, so key them by `custom_id`.
- **Effort:** set `output_config.effort` explicitly per step (low for query generation, higher for rule checks).
- **Forced tool choice** is not supported on the newest Opus and Sonnet models (it returns a 400). Use structured outputs instead.
- **Refusals:** always check `stop_reason` before reading the answer, and handle `"refusal"`.
- **Record and replay:** tests and CI replay saved responses, so they cost $0 and give the same result every time.

10. Evaluation plan (what makes this project stand out)

Evaluation is the most valuable skill this project teaches. Every claim on your resume will come from this section.

10.1 Metrics in plain words

Metric	Plain meaning	Measured on
nDCG@10	How good the top 10 are, giving more credit for putting the best trials first and for eligible (2) over excluded (1)	TREC qrels
Precision@10	Of the top 10 trials, how many are relevant	TREC qrels
Recall@100	Of all relevant trials, how many made it into the top 100 (does retrieval miss things?)	TREC qrels
MRR	How high the first good trial appears, on average	TREC qrels
Eligible vs. excluded accuracy	Among judged trials, does Beacon's category match the experts'?	TREC qrels

Metric	Plain meaning	Measured on
Rule accuracy	Are individual rule judgments right?	Your 150–200 hand-labeled judgments
Evidence faithfulness	Share of judgments whose cited sentences really support them (the inverse is your hallucination rate)	Verifier plus a hand-checked sample
Invalid citation rate	Citations to sentences that don't exist (should be 0%)	Code check
Readability	Reading grade level of patient summaries (aim for about grade 8)	textstat library
Cost and latency	Dollars and seconds per patient, p50 and p95	Audit log

10.2 Making the numbers trustworthy

- **Calibrate your LLM judge.** Hand-label about 60 judgment/evidence pairs, run the judge on the same pairs and report agreement with Cohen's kappa (κ). Above 0.6 is generally considered substantial. Without this step, LLM-judge numbers are just opinions.
- **Use confidence intervals.** With 50 test patients, a 2-point difference may be noise. Use bootstrap confidence intervals and paired significance tests (the ranx library has both).
- **Keep the split clean.** Tune on 2021, report on 2022, and never look at 2022 while tuning.
- **Slice the results.** Report metrics by patient age group and sex to check that it works equally well for everyone.
- **Compare with TrialGPT** using the numbers reported in its paper (same benchmark). If your budget allows, also run its open-source code yourself.

10.3 Evaluation in CI

Every pull request runs a small evaluation (for example 10 patients) using recorded LLM responses, so it is free and repeatable. The build fails if nDCG@10 drops by more than 2 points or the invalid-citation rate rises above 0 compared with the main branch. The full benchmark runs on demand through the Batches API.

10.4 Tables you will publish in the README

Config (TREC 2022, held-out)	nDCG@10	P@10	Recall@100	Cost / patient
BM25	—	—	—	$0
Hybrid + rerank	—	—	—	$0
Full Beacon (Haiku 5.5)	—	—	—	—
Full Beacon (final model)	—	—	—	—
TrialGPT (reported in its paper)	—	—	—	n/a

Fill these in only with numbers you measured. Never estimate a result.

11. Production patterns (the backend depth)

Pattern	Plain meaning	Where in Beacon
Idempotency	Doing the same thing twice has the same effect as once	Retried POSTs (Redis key); re-delivered Kafka messages (unique constraints)
At-least-once + idempotent consumers	Kafka may deliver a message twice, so make processing safe to repeat	Match workers
Transactional outbox	Save data and the event-to-send in one database transaction, then publish	Gateway → Kafka
Retries with backoff + jitter	Wait longer after each failure, with randomness, so services don't stampede	LLM calls, retrieval calls
Dead-letter queue (DLQ)	Messages that keep failing are parked for inspection instead of blocking others	match.dlq
Token-bucket rate limiting	Each user gets N requests per minute, refilled steadily	Gateway, plus a shared LLM tokens-per-minute limit across all workers
Timeouts and graceful degradation	If a part is down, keep working with less	Reranker down → serve RRF results only, flagged
Durable workflows	Save progress so crashes don't lose work	LangGraph Postgres checkpoints
Budgets	Hard ceilings on tokens and dollars	Per match and per day

Crash test for your demo: start a match, kill the worker container halfway through, start it again and show the match finishing with no duplicates. It's a 20-second clip that shows real distributed-systems understanding.

12. Observability

- **One trace per request:** gateway → Kafka (trace context travels in message headers) → worker → every LangGraph node → every LLM call and retrieval call. You open one request and see the whole journey with timings.
- **LLM visibility:** Phoenix shows every prompt, response, token count and latency. Your own `llm_calls` table records dollars per call.
- **Structured logs** (JSON, via structlog) that include the trace ID, so logs and traces connect.
- **A small "System" page in the console:** requests today, p50/p95 latency, cost per patient, cache hit rate and failed jobs in the DLQ.

13. Cost plan and controls

Assumptions: 20 trials checked per patient, about 5,000 input and 1,500 output tokens per trial check, before prompt-caching savings. These are estimates; your audit log will give the real numbers.

Run	Claude Haiku 5.5	Claude Sonnet 5.5	Claude Opus 5.5
Price (input / output per 1M tokens)	$0.10 / $0.50	$2 / $10	$4 / $20
One patient (20 trials)	about $0.03	about $0.50	about $1.00
Full TREC run (125 patients)	about $3	about $60 (about $30 with Batches)	about $125 (about $60 with Batches)

Recommended budget: about $30–80 total

- Develop and tune on **Haiku 5.5** (about $3 per full run, so you can iterate freely).
- Run the **final headline evaluation once** on Sonnet 5.5 through the Batches API (about $30). Opus 5.5 is the most capable option if you want the strongest final numbers and the budget allows.
- Embeddings, BM25 and reranking all run locally for **$0**.
- Controls: a monthly spend limit in the Anthropic Console, a per-match and per-day budget in code, an exact-match response cache in Redis, recorded responses for tests and CI, and prompt caching.

14. Repository layout

```
beacon/
|-- services/
|   |-- gateway/          FastAPI: auth, rate limits, /v1/matches, SSE  (grows from your router.py)
|   |-- retrieval/        FastAPI: hybrid search + rerank
|   |-- match_worker/     LangGraph pipeline, Kafka consumer
|   `-- ingestion/        TREC loader now; ClinicalTrials.gov daily sync in phase 2
|-- libs/
|   |-- beacon_core/      settings, schemas, db, kafka, redis, telemetry, logging
|   `-- llm/              Claude wrapper: structured outputs, caching, budgets, cost, record/replay
|-- evals/                TREC loaders, metrics (ranx), judges, hand labels, ablations, reports
|-- web/                  Next.js console
|-- deploy/               compose.yaml, service configs, init SQL
|-- docs/                 ARCHITECTURE.md, adr/0001-..., demo assets
|-- tests/                unit + integration (testcontainers)
`-- .github/workflows/    ci.yml (lint, types, tests, eval gate, image builds)
```

What happens to your current code

Your `router.py` and `service.py` become the gateway's "ask about these results" chat endpoint (`/v1/matches/{id}/chat`). Keep your good choice of opening the Claude stream before returning the response, so errors become proper HTTP codes. Fix three things: move conversation history out of process memory into Redis/Postgres, save the user and assistant turns together only after success, and send typed SSE events (token, error, done) instead of writing errors into the text.

15. Build plan

15.1 The 8-day sprint (November 7–14)

Day	Focus	Tasks	Done when
Nov 7	Foundations	WSL2 + Docker; uv workspace; Compose with Postgres, Redis, Qdrant, Redpanda, Phoenix (with healthchecks); beacon_core settings/logging/db; gateway skeleton from your code; CI (ruff, mypy, pytest); download TREC via ir_datasets	docker compose up is all green; CI passes; you can print a TREC patient
Nov 8	Ingestion	Parse 375K trials into Postgres (structured fields); rule splitter with unit tests; BM25 + dense vectors into Qdrant (run overnight)	Qdrant holds 375K points; 10 trials spot-checked by hand
Nov 9	Retrieval + first numbers	Retrieval service (hybrid, RRF, filters, rerank); evaluation harness with ranx; ablation v1 (tune on 2021, test on 2022)	Ablation table with real numbers committed. Your first resume number.

Day	Focus	Tasks	Done when
Nov 10	LLM layer	llm lib (structured outputs, caching, cost metering, budgets, record/replay); patient profiler; query generation; rule checker with sentence citations; scoring	One TREC patient → ranked top 20 with cited rule judgments, from a script
Nov 11	Pipeline	LangGraph graph; match_worker on Kafka; progress events; SSE; idempotency, retries, DLQ; Postgres checkpoints	POST /v1/matches streams live progress; killing a worker mid-run still finishes the job
Nov 12	Evaluation depth	Full TREC run on Haiku; graded nDCG; hand-label 150 rule judgments; verifier and faithfulness; judge calibration (kappa); bootstrap CIs; CI eval gate	Evaluation report generated automatically by one command
Nov 13	Production polish	OpenTelemetry end to end; Phoenix traces; rate limits; graceful degradation; final headline run via Batches; write ADRs	One trace shows the whole journey; headline numbers final
Nov 14	Console + story	Next.js page (paste note → live progress → ranked trials → rule table with highlighted evidence → patient summary); README; 2-minute demo video; resume bullets	A stranger can clone, run and understand it
Nov 15	Apply	Submit applications with the GitHub link. Don't wait for perfect.	Applications sent

15.2 Weeks 2–4 (after the deadline)

- **Live mode:** daily ClinicalTrials.gov sync through Kafka (trials.updated), a recruiting-only filter and distance to trial sites.
- **MCP server** exposing search_trials and check_eligibility, so Claude Desktop or Claude Code can use Beacon.
- **Patient experience:** plain-language and multilingual summaries, scored for reading level.
- **Better retrieval:** MedCPT biomedical embeddings, rule-level chunks with contextual headers, and per-query fusion weights.
- **Reviewer UI:** mark judgments wrong, with corrections growing the evaluation set.
- **Write-up:** a blog post with the ablation results. It's great interview material and a shareable link.

15.3 If you fall behind: cut in this order

Console polish → live SSE progress (poll instead) → hand-label set size (100 instead of 150) → tracing across Kafka (keep it inside services). **Never cut:** the TREC numbers, the citations, tests and CI, or the README.

16. Learning roadmap: everything you need to learn

How to learn this efficiently

- **Learn just in time.** Study each topic the day before you build with it, not weeks ahead.
- **Do a 30-minute toy first.** Before adding a tool to Beacon, make a tiny throwaway script that uses it alone.
- **Explain it back.** After each topic, write three sentences in docs/notes.md explaining it simply. If you can't, you don't know it yet, and this file becomes your interview prep.
- **Use AI to explain, not to replace understanding.** Ask "why" until the answer is obvious to you.

Tier 1: Foundations (about 10 hours, refresh what you don't know)

Topic	What you must be able to do	Hours
Python async (asyncio)	async/await, gather, timeouts, TaskGroup; why async helps I/O-heavy services	3
Type hints + Pydantic v2	Models, validation, nested schemas, pydantic-settings for config	2
Docker + Compose	Images vs. containers, volumes, networks, healthchecks, depends_on, logs	3
SQL basics	Joins, indexes, transactions, unique constraints	2
Git + GitHub flow	Branches, pull requests, meaningful commits	1

Tier 2: Core AI engineering (about 25 hours, the heart of your resume)

Topic	What you must be able to do	Hours
LLM APIs (Claude)	Messages, system prompts, streaming, structured outputs, tool use, prompt caching, Batches, token counting, cost math	4
Prompting for extraction and classification	Clear label definitions, examples, evidence-citing output, abstaining (“insufficient information”)	2
Embeddings	What a vector is; cosine similarity; bi-encoder vs. cross-encoder; sequence-length limits; choosing a model	2
Vector databases (Qdrant)	Collections, HNSW index, payload filters, named vectors, sparse vectors, quantization, Query API with prefetch and fusion	3
Keyword search (BM25)	Term frequency, IDF, tokenization; why exact matches matter in medicine	1.5
Hybrid search + RRF	Fusing rankings; the RRF formula; weighting	1
Reranking	Cross-encoders, the cost/latency tradeoff, how many to rerank	1
Chunking	Structure-aware chunks, rule-level chunks, contextual headers	1
IR evaluation	qrels, nDCG, P@k, Recall@k, MRR; train/test split; ranx or ir_measures	3
LLM evaluation	LLM-as-judge, rubrics, Cohen's kappa calibration, faithfulness, bootstrap confidence intervals	3
Agents with LangGraph	StateGraph, nodes, conditional edges, Send for parallel fan-out, Postgres checkpointer, interrupts	4
Hallucination control	Citations, verification steps, abstention, budgets	1

Tier 3: Backend and distributed systems (about 14 hours)

Topic	What you must be able to do	Hours
FastAPI	Routers, dependencies, async endpoints, error handling, SSE streaming, OpenAPI docs	3
PostgreSQL + SQLAlchemy 2 (async) + Alembic	Models, sessions, migrations, transactions, upserts	3
Redis	Strings, hashes, TTL, SET NX locks, a Lua token bucket, pub/sub	2
Kafka concepts + aiokafka	Topics, partitions, keys, consumer groups, offsets, at-least-once delivery, idempotency, retries, DLQ, outbox	4

Topic	What you must be able to do	Hours
Resilience patterns	Timeouts, exponential backoff with jitter (tenacity), circuit breakers, graceful degradation	2

Tier 4: Observability and engineering quality (about 8 hours)

Topic	What you must be able to do	Hours
OpenTelemetry	Traces, spans, attributes, context propagation (including through Kafka headers)	2
Arize Phoenix	Send traces, read LLM spans, find slow or expensive calls	1
Structured logging	JSON logs with trace IDs (structlog)	0.5
pytest	Fixtures, async tests, mocking HTTP (respx), testcontainers for real Postgres/Redis/Kafka	2.5
GitHub Actions	Jobs, caching, service containers, secrets, required checks	1.5
ruff, mypy, uv	Linting, type checking, dependency locking	0.5

Tier 5: The medical domain (about 5 hours)

Topic	What you must be able to do	Hours
Clinical trial basics	Phases I–IV, inclusion/exclusion criteria, NCT IDs, recruiting status, how a ClinicalTrials.gov record is structured	1.5
Medical vocabulary	Common terms in TREC topics: HbA1c, eGFR/creatinine, ECOG performance status, metastasis, biomarkers. Look each up as you meet it.	1.5
Key papers	The TrialGPT paper (Nature Communications, 2024) and the TREC Clinical Trials 2021 track overview	2

Tier 6: Frontend, thin (about 5 hours)

React and Next.js basics (components, state, fetching data), EventSource for live server-sent events, and Tailwind for quick styling. Keep it to one or two pages.

What NOT to learn right now

Kubernetes, Terraform, fine-tuning large models, GraphRAG, LangChain internals and advanced frontend. They are all valuable later, but none of them improves this project's headline before November 15.

17. Testing, CI/CD and containers

Layer	What to test	Tools
Unit	Rule splitter, RRF, scoring, citation validation, cost math, rate limiter	pytest
Integration	Real Postgres, Redis, Qdrant and Kafka in throwaway containers; idempotency; DLQ path	pytest + testcontainers
LLM-dependent	Pipeline runs with recorded responses (no API cost, deterministic)	Your record/replay layer
Evaluation gate	10-patient TREC subset; fail on regression	evals/ + ranx
End to end	docker compose up → POST a match → receive results	Smoke test script

CI pipeline: ruff → mypy → unit tests → integration tests → evaluation gate → build Docker images → push to GitHub Container Registry on main. **Docker practices:** multi-stage builds with uv, a non-root user, slim base images, pinned versions and healthchecks on every service.

18. Resume, README and interview prep

18.1 Resume bullets (fill in only with numbers you measured)

- Built **Beacon**, an open-source clinical-trial matching engine (Python, FastAPI, LangGraph, Claude, Qdrant, Kafka, Redis, PostgreSQL, OpenTelemetry, Docker). It searches 375K trials and judges each eligibility rule with sentence-level evidence, reaching **nDCG@10 of X** on the held-out TREC Clinical Trials 2022 benchmark (vs. **Y** for a BM25 baseline).
- Designed hybrid retrieval (BM25 + dense embeddings, RRF fusion, cross-encoder reranking, LLM query generation), improving **Recall@100 from A to B**; tuned on TREC 2021 and reported on 2022 to avoid test-set overfitting.
- Built an evaluation pipeline (rule-level accuracy, evidence faithfulness, an LLM judge calibrated to human labels at **κ = X**) enforced as a CI regression gate; **X%** of judgments backed by verified citations.
- Engineered an event-driven backend (Kafka consumer groups, idempotent workers, retries with a dead-letter queue, durable LangGraph checkpoints) at **p95 X s** and **$Y per patient**, with zero lost jobs under worker-crash tests.

18.2 README checklist

- One-sentence pitch plus a 2-minute demo GIF or video at the top
- The architecture diagram
- The headline results table and the ablation table, with confidence intervals
- “Run it in one command” instructions, tested from a clean clone
- Design decisions with links to the ADRs, and known limitations (be honest)
- Medical disclaimer and data sources (TREC, ClinicalTrials.gov)

18.3 Demo video script (2 minutes)

1. The problem in one sentence, with Sarah's story (15 s).
2. Paste a TREC patient note and watch live progress (30 s).
3. Open the top trial: the rule table, then click a rule to highlight its evidence sentence (30 s).
4. Show the patient summary and the missing-facts list (15 s).
5. Kill a worker mid-run and show the job resume (15 s).
6. Show the results table versus BM25 and TrialGPT, then the cost per patient (15 s).

18.4 Questions you must be able to answer

Question	Answer outline
Why hybrid search instead of only vectors?	Exact biomedical tokens plus synonyms; point to your ablation numbers
What is RRF, and why k = 60?	Rank-based fusion that needs no score normalization; k dampens the top ranks; you tested alternatives
Why rerank only the top 100?	A cross-encoder scores each pair separately, so cost and latency grow with k; you measured the curve
How do you know it isn't hallucinating?	Sentence citations, code validation, the verifier, the faithfulness rate, hand-checked samples
How did you validate the LLM judge?	Cohen's kappa against your own labels on about 60 pairs
Why tune on 2021 and test on 2022?	To avoid overfitting to the test set; the honest generalization number
What happens if a worker crashes?	Kafka redelivers; offsets are committed after checkpoints; writes are idempotent
Why Kafka instead of a background task?	Durability, replay, consumer-group scaling, backpressure, retries/DLQ
Why both Redis and Postgres?	Hot, short-lived data vs. the durable source of truth
How did you keep costs down?	Haiku for development, prompt-cache ordering, Batches, exact caching, budgets, local embeddings
How would you scale to 10M trials or 1,000 requests/s?	Shard Qdrant, more partitions and workers, precompute patient-independent work (rule parsing), cache
What would a hospital need?	HIPAA compliance, EHR integration (FHIR), clinical validation, bias audits, monitoring
What failed, and what surprised you?	Prepare a real story from your own logs. This question is always asked.
Does it work equally well for everyone?	Show metrics sliced by age group and sex

18.5 Architecture Decision Records to write (one page each)

0001 Qdrant vs. pgvector · 0002 Kafka (Redpanda) vs. Redis Streams · 0003 LangGraph with the raw SDK vs. LangChain · 0004 trial-level vs. rule-level indexing · 0005 model choice and cost · 0006 the 2021/2022 tuning/testing split. Each covers the context, the options, the decision and the consequences. They double as interview prep.

19. Risks and how to handle them

Risk	Mitigation
Running out of time	Follow the cut order in 15.3; apply on Nov 15 regardless
Corpus too big for the laptop	One dense vector per trial, quantization, run embedding overnight, or use Colab; subset only as a last resort (it changes recall numbers)
API costs creeping up	Haiku for development, a spend limit, budgets in code, record/replay, Batches for big runs
Unfamiliar medical terms	TREC notes are readable; keep a glossary; ask Claude to explain terms (not to label your evaluation data)
Overfitting to the benchmark	Strict 2021-tune / 2022-test split
A biased LLM judge	Kappa calibration; use a different model as the judge than the one being judged
Windows tooling friction	Work inside WSL2; keep commands in a Makefile
Scope creep	Nothing from Section 15.2 before the core and the numbers are done
Code you can't explain	Type the core parts yourself; keep docs/notes.md; do mock project deep-dives

20. Medical, safety and ethics notes

- Beacon is **decision support**, not medical advice. Say so in the console, the API responses and the README.
- Use **synthetic data only** (TREC topics). Never put real patient records in this project.
- Using real data later would require privacy-law compliance (HIPAA in the US, provincial laws such as PHIPA in Ontario) and ethics-board approval. That is out of scope.
- Don't claim clinical validity. Claim benchmark performance with exact numbers and confidence intervals.
- Report fairness slices, and state the limits honestly (TREC patients are synthetic; the trial snapshot is from 2021).

Appendix A. Glossary in plain words

Term	Meaning
API	A way for programs to talk to each other over the network.
Async (asyncio)	Python code that can wait for many network calls at once instead of one at a time.
ADR	Architecture Decision Record: a one-page note explaining a design choice.
BM25	Classic keyword search scoring; rewards rare words that appear in a document.
Bi-encoder	Embeds the query and documents separately: fast, used for first-stage search.
Bootstrap CI	Resampling your results many times to see how much a score could vary by chance.
Checkpoint	Saved progress of a workflow, so it can resume after a crash.
Chunk	A piece of a document stored and searched separately.
CI/CD	Automatically testing (and shipping) code on every change.
Cohen's kappa (κ)	Agreement between two labelers, corrected for chance agreement.
Consumer group	A set of workers sharing one Kafka topic; each message goes to one of them.
Container	A packaged, isolated program with everything it needs (Docker).

Term	Meaning
Cosine similarity	How closely two vectors point in the same direction, meaning how similar two texts are.
Cross-encoder	Reads the query and document together for accurate scoring; slower, used for reranking.
Dead-letter queue (DLQ)	Where messages that keep failing are parked for inspection.
Embedding	A list of numbers representing the meaning of a text.
Exclusion / inclusion criteria	Trial rules you must not have / must have.
Faithfulness	Whether an AI's claim is actually supported by its cited evidence.
Hallucination	An AI stating something unsupported or false.
HNSW	The graph index vector databases use to find nearest vectors fast.
Hybrid search	Combining keyword search and meaning (vector) search.
Idempotent	Safe to repeat: doing it twice equals doing it once.
Kafka	A durable log of messages that services publish to and read from.
LangGraph	A library for multi-step AI workflows as graphs with saved state.
Latency p50 / p95	The time within which 50% / 95% of requests finish.
LLM-as-judge	Using an LLM to grade outputs; it must be calibrated against humans.
MCP	Model Context Protocol: a standard way to give AI assistants tools.
nDCG@10	Ranking quality of the top 10, rewarding the best results at the top.
NCT ID	A trial's unique ID on ClinicalTrials.gov (for example NCT01234567).
Observability	Being able to see what your system is doing from the outside (traces, logs, metrics).
Offset	A consumer's position in a Kafka topic.
OpenTelemetry	The open standard for collecting traces and metrics.
Outbox pattern	Saving data and the event-to-send in one transaction, then publishing.
Partition	A slice of a Kafka topic; messages with the same key stay in order.
Phoenix	An open-source UI for tracing and evaluating LLM apps.
Prompt caching	Reusing an already-processed prompt prefix for cheaper, faster calls.
qrels	Relevance labels: the answer key for search evaluation.
Qdrant	A vector database for dense and sparse vectors with filters.
RAG	Retrieval-augmented generation: find relevant text first, then have the LLM answer using it.
Rate limiting (token bucket)	Allowing N requests per period, refilled steadily.
Recall@k	Share of all relevant items that appear in the top k.
Redis	A very fast in-memory database for caches, counters and locks.
Reranker	A second, more accurate model that reorders the top search results.
RRF	Reciprocal Rank Fusion: merges several rankings using 1/(k + rank).
SSE	Server-sent events: the server pushes live updates to the browser.
Structured outputs	Forcing the LLM to return JSON that matches your schema.
Trace / span	A trace is one request's whole journey; spans are its timed steps.

Term	Meaning
TREC	A NIST research competition with standard benchmarks and expert labels.
Vector database	A database built to find the stored vectors closest to a query vector.

Appendix B. Where to learn each thing

Topic	Resource
Claude API	Claude developer docs at platform.claude.com/docs (messages, structured outputs, prompt caching, Batches)
Agent design	Anthropic engineering posts “Building effective agents” and “Introducing Contextual Retrieval” (anthropic.com)
FastAPI	fastapi.tiangolo.com: the tutorial plus the “Advanced User Guide” sections on dependencies and streaming
Qdrant + FastEmbed	qdrant.tech/documentation: hybrid queries, sparse vectors, filtering, quantization
LangGraph	Official LangGraph docs (docs.langchain.com): graphs, Send, persistence/checkpointers, interrupts
IR fundamentals	Introduction to Information Retrieval (Manning, Raghavan, Schütze), free at nlp.stanford.edu/IR-book; chapters 6 and 8
IR evaluation in Python	ir-datasets.com (TREC data loading) and the ranx library on GitHub (metrics, fusion, significance tests)
Benchmark	trec.nist.gov: Clinical Trials track 2021/2022 overview papers and data
Prior work	TrialGPT: github.com/ncbi-nlp/TrialGPT and the Nature Communications paper (2024)
Trial data	ClinicalTrials.gov API documentation (clinicaltrials.gov/data-api/api)
Kafka and distributed systems	Redpanda docs and quickstart; Designing Data-Intensive Applications (Kleppmann), chapter 11 on streams
Redis	redis.io/docs: data types, expiration, Lua scripting, pub/sub
PostgreSQL + SQLAlchemy	SQLAlchemy 2.0 docs (async ORM) and the Alembic tutorial
Observability	opentelemetry.io/docs (Python) and the Arize Phoenix docs
Testing and CI	docs.pytest.org, testcontainers-python, GitHub Actions docs
Docker	docs.docker.com: Compose file reference, healthchecks, multi-stage builds

Next step: Day 1 (Foundations). Set up WSL2 and Docker, create the uv workspace and Compose stack, turn your existing chat code into the gateway skeleton, and get CI green.